Base de Datos — Diseño Seguro y Gestión de Usuarios

Contexto y Rediseño de Seguridad

Al pasar el juego de un entorno local con XAMPP a una exposición real en internet, el esquema original de la base de datos resultó insuficiente. El problema principal era el uso de IDs numéricos secuenciales (1, 2, 3...), lo que permitía que cualquier usuario pudiera adivinar y acceder a otras partidas simplemente cambiando el número en la URL (ataque de enumeración).

Para mitigar esto, implementé un diseño de seguridad por diseño:

- Código público aleatorio: Sustituí el ID por un código de 8 caracteres únicos (ej: A3F9K2BX) generado con MD5 y UUID, imposible de predecir.
- Tokens SHA-256 por jugador: Cada partida genera dos tokens únicos e internos. El servidor valida estos tokens mediante `hash_equals()` antes de aceptar movimientos, garantizando que solo el jugador legítimo pueda mover sus fichas.
- Tabla de movimientos: Añadí una tabla para registrar cada jugada con su *timestamp* y tipo de acción (captura o coronación), asegurando trazabilidad completa para auditoría.

Implementación del Esquema y Usuarios

Para asegurar una configuración limpia, desarrollé un script integral llamado `schema.sql` que automatiza la creación de la base de datos `damas`, las tablas, los procedimientos almacenados y la jerarquía de usuarios.El despliegue se realizó directamente en el servidor MySQL (10.1.0.30) forzando la creación de la estructura optimizada para el entorno web.

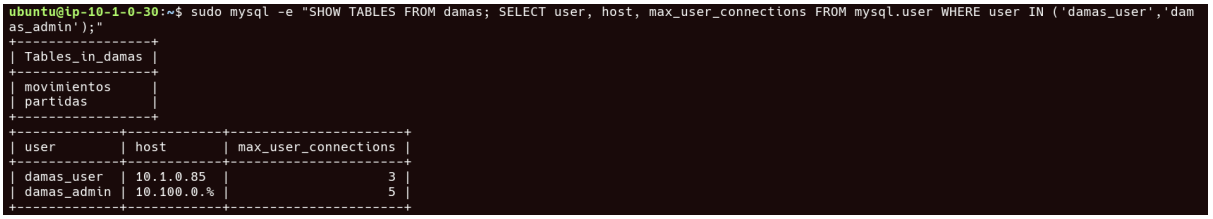
```
ubuntu@ip-10-1-0-30:~$ nano schema.sql
ubuntu@ip-10-1-0-30:~$ sudo mysql -v < schema.sql
--------------
DROP DATABASE IF EXISTS damas
--------------

--------------
CREATE DATABASE damas
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci
--------------

--------------
CREATE TABLE partidas (
  id              INT UNSIGNED    NOT NULL AUTO_INCREMENT,
  codigo          CHAR(8)         NOT NULL,
  token_rojo      CHAR(64)        NOT NULL,
  token_negro     CHAR(64)        NOT NULL,
  apodo_rojo      VARCHAR(30)     NOT NULL DEFAULT 'Jugador Rojo',
  apodo_negro     VARCHAR(30)     NOT NULL DEFAULT 'Jugador Negro',
```

Verificación de Tablas y Roles

Tras la ejecución, verifiqué que la base de datos contuviera las tablas necesarias y que los usuarios estuvieran limitados por el principio de mínimo privilegio.



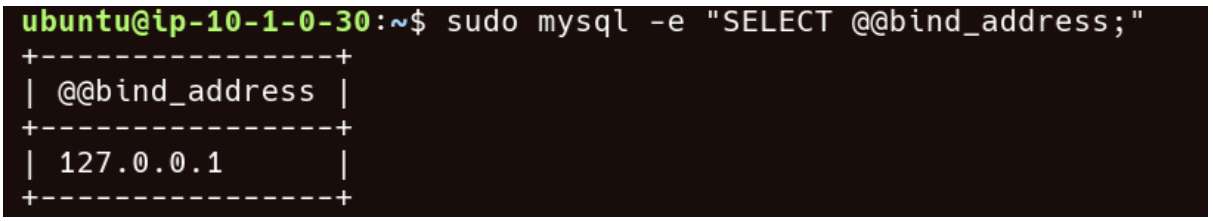
(Evidencia de las tablas `movimientos` y `partidas` creadas, junto con los usuarios `damas_user` y `damas_admin` configurados con sus respectivas restricciones de conexión.)

Configuración de Red (MySQL Bind Address)

Un paso crítico que a menudo se olvida en local es que MySQL, por defecto, solo escucha en `127.0.0.1`. Para que el servidor Apache (10.1.0.85) pudiera conectar, tuve que modificar la configuración del servidor para que escuchara en su interfaz de red privada (10.1.0.30).

Detección del Bloqueo Local

Primero confirmé que el servidor estaba cerrado a conexiones externas.



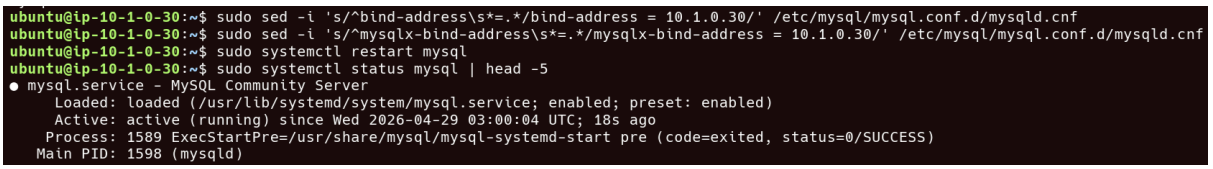
(Muestra el `@@bind_address` inicial en `127.0.0.1`, lo cual impedía la conexión desde Apache.)



(Uso de `grep` para localizar las líneas de configuración en `/etc/mysql/mysql.conf.d/mysqld.cnf`.)

Aplicación del Cambio y Reinicio

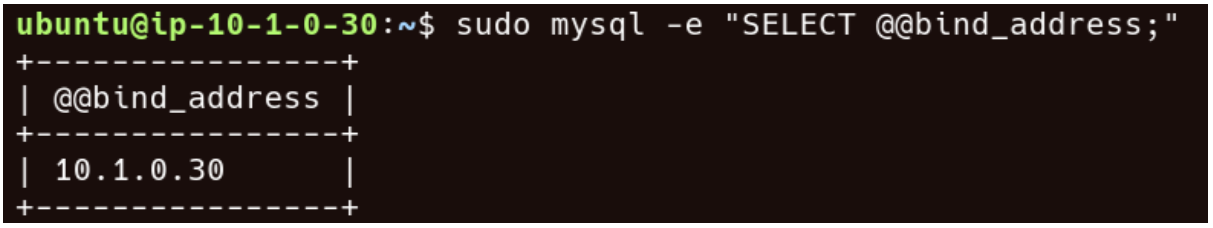
Utilicé el comando `sed` para actualizar la dirección de escucha y reinicié el servicio para aplicar los cambios.



(Comandos sed aplicados, reinicio del servicio mediante `systemctl restart mysql` y verificación de que el servicio está activo.)

Validación Final de Red

Finalmente, comprobé que MySQL estaba escuchando en la IP correcta de la red de AWS.



(Resultado exitoso: el `@@bind_address` ahora refleja `10.1.0.30`, permitiendo el tráfico desde el servidor web.)

Arquitectura de Usuarios

El sistema queda blindado bajo el siguiente esquema de accesos:

Usuario	Acceso desde	Función	Restricción Clave
`damas_user`	10.1.0.85 (Apache)	Aplicación web	Máximo 3 conexiones simultáneas.
`damas_admin`	10.100.0.% (VPN)	Mantenimiento	Solo accesible vía VPN, nunca desde internet.
`root`	`localhost`	Emergencias	Solo accesible físicamente desde el servidor.

Conclusión

Con la **Issue** cerrada, la base de datos ha pasado de ser un almacén local inseguro a una infraestructura de producción. Tenemos un esquema que protege la identidad de los jugadores mediante tokens SHA-256 , un registro de auditoría completo y un motor de base de datos configurado para comunicarse de forma segura dentro de la red privada de AWS